

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand
9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply

10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply
----	--	-------------

Code of Conduct:

*I M.Devi Srilatha(192525236) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q1. Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

1. Introduction

The **CREATE TABLE** statement is one of the most important SQL Data Definition Language (DDL) commands. It is used to create a new table in a database by defining the table name, columns, data types, and constraints. A properly designed table ensures data accuracy, integrity, and consistency.

Errors in a CREATE TABLE statement may prevent the table from being created or may result in an improperly designed database. Therefore, identifying and correcting syntax and constraint errors is an essential SQL debugging skill.

Definition of CREATE TABLE

The **CREATE TABLE** command is used to create a new table in a relational database.

```
CREATE TABLE table_name
(
    column1 datatype constraints,
    column2 datatype constraints,
    ...
);
```

3. Importance of Constraints

Constraints ensure that only valid data is stored in the table.

Common Constraints:

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE
- CHECK
- DEFAULT
-

4. Incorrect SQL Statement

```
CREATE TABLE Student
(
    StudentID INT
    Name VARCHAR(50),
    Age INT
    PRIMARY KEY StudentID
);
```

5. Error Identification

Error 1

Missing comma after StudentID.

Incorrect:

```
</> SQL

StudentID INT
Name VARCHAR(50)
```

Correct:

```
</> SQL

StudentID INT,
Name VARCHAR(50)
```

Error 2

Missing comma after Age.

Incorrect:

```
</> SQL

Age INT
PRIMARY KEY
```

Correct:

```
</> SQL

Age INT,
PRIMARY KEY
```

Error 3

Incorrect PRIMARY KEY syntax.

Incorrect:

```
</> SQL

PRIMARY KEY StudentID
```

Correct:

```
</> SQL

PRIMARY KEY(StudentID)
```

Error 4

StudentID should be NOT NULL because it is the primary key.

Correct:

```
</> SQL

StudentID INT NOT NULL
```

6. Correct SQL Statement

```
CREATE TABLE Student
(
    StudentID INT NOT NULL,
    StudentName VARCHAR(50),
    Age INT,
    PRIMARY KEY(StudentID)
);
```

StudentID INT NOT NULL

- Integer datatype.
- Cannot contain NULL values.
- Used to uniquely identify each student.

StudentName VARCHAR(50)

- Stores student name.
- Maximum 50 characters.

Age INT

- Stores student's age.

PRIMARY KEY(StudentID)

- Ensures uniqueness.
- Does not allow duplicate values.
- Does not allow NULL values.

8. Sample Table Structure

StudentID StudentName Age

101	Ravi	20
102	Priya	21
103	Kiran	22

9. Execution Result

```
Table created successfully.
```

10. Advantages of Constraints

- Maintains data integrity.
- Prevents duplicate records.
- Prevents NULL values in important columns.
- Improves database consistency.
- Makes relationships easier.

11. Applications

- College Management System
- Hospital Database
- Banking System
- Employee Management
- Railway Reservation
- Library Management

12. Common Mistakes

- Missing commas.
- Wrong datatype.
- Incorrect constraint syntax.
- Duplicate column names.
- Missing PRIMARY KEY.

14. Conclusion

The CREATE TABLE statement is the foundation of database design. Correct syntax, proper datatypes, and appropriate constraints ensure reliable and consistent data storage. Debugging CREATE TABLE statements improves SQL programming skills and helps create efficient databases.

Q2. Debug an SQL Query that Incorrectly Uses Aggregate Functions Without GROUP BY

1. Introduction

Aggregate functions perform calculations on multiple rows and return a single result. When aggregate functions are used with normal columns, SQL requires the **GROUP BY** clause to group similar records.

Without GROUP BY, SQL generates an error because it cannot determine how to group non-aggregated columns.

2. Aggregate Functions

Common aggregate functions include:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

3. GROUP BY Definition

GROUP BY divides rows into groups so aggregate functions can calculate results for each group separately.

4. Incorrect Query

```
SELECT Department,  
AVG(Salary)  
FROM Employee;
```

5. Error

Department is a normal column.

AVG() is an aggregate function.

SQL does not know how to calculate the average salary for each department.

6. Correct Query

```
SELECT Department,  
AVG(Salary) AS AverageSalary  
FROM Employee  
GROUP BY Department;
```

7. Sample Employee Table

EmpID	Name	Department	Salary
1	Ram	HR	30000
2	Asha	HR	40000
3	John	IT	60000
4	Priya	IT	70000
5	David	Sales	50000

8. Output

Department	AverageSalary
HR	35000
IT	65000
Sales	50000

9. Explanation

GROUP BY Department

Groups employees according to department.

AVG(Salary)

Calculates average salary within each department.

AS AverageSalary

Renames the output column.

10. Working Process

Employee Table

↓

GROUP BY Department

↓

HR Group

30000

40000

↓

Average = 35000

↓

IT Group

60000

70000

↓

Average = 65000

↓

Sales Group

50000

↓

Average = 50000

11. Advantages of GROUP BY

- Produces summarized reports.
- Simplifies data analysis.
- Improves readability.
- Useful in business intelligence.
- Supports statistical calculations.

12. Real-Life Applications

- Department-wise salary report.
- Product-wise sales report.
- Student-wise average marks.
- Branch-wise bank transactions.
- Monthly revenue analysis.

13. Common Errors

- Forgetting GROUP BY.
 - Misspelling column names.
 - Using aggregate functions in WHERE instead of HAVING.
 - Incorrect grouping.
-

15. Conclusion

Aggregate functions summarize data, while GROUP BY organizes rows into meaningful groups. Correct usage enables accurate reporting, improves query performance, and supports effective business decision-making.

Q3. Correct a Faulty SELECT Query Where Column Names Are Misspelled or Improperly Referenced

1. Introduction

The **SELECT** statement is the most frequently used SQL command. It retrieves data from one or more tables. If column names are misspelled or incorrectly referenced, SQL cannot find the required column and returns an error.

Understanding how to identify and correct such errors is essential for writing reliable SQL queries.

2. SELECT Statement

The **SELECT** statement retrieves data from database tables.

Syntax

```
SELECT column1, column2
FROM table_name;
```

3. Incorrect Query

```
SELECT StudntName,
Age
FROM Student;
```

4. Error Identification

The column name **StudntName** is incorrect.

Correct column name:

```
StudentName
```

5. Correct Query

```
SELECT StudentName,
Age
FROM Student;
```

6. Sample Student Table

StudentID	StudentName	Age
101	Asha	20
102	Rahul	21
103	Priya	22

7. Output

StudentName	Age
Asha	20
Rahul	21
Priya	22

8. Explanation

SELECT

Retrieves specified columns.

StudentName

Displays student names.

Age

Displays age.

FROM Student

Specifies the source table.

9. Other Common Errors

Wrong Table Name

```
SELECT Name  
FROM Students;
```

Correct:

```
SELECT Name  
FROM Student;
```

Missing Comma

Incorrect

```
SELECT StudentName Age  
FROM Student;
```

Correct

```
SELECT StudentName, Age  
FROM Student;
```

Invalid Alias

Incorrect

```
SELECT StudentName AS  
FROM Student;
```

Correct

```
SELECT StudentName AS Name  
FROM Student;
```

10. Advantages of Correct Queries

- Faster execution.
- Accurate results.
- Prevents runtime errors.
- Easy to maintain.
- Improves readability.

11. Real-Life Applications

- Student information systems.
- Banking databases.
- Hospital records.
- E-commerce product lists.
- Employee management systems.
- Library databases.

12. Common SQL Errors

- Misspelled column names.
 - Wrong table names.
 - Missing commas.
 - Incorrect aliases.
 - Case-sensitive object names (in some DBMSs).
-

14. Conclusion

The **SELECT** statement is fundamental to SQL. Correct column references, proper syntax, and accurate table names ensure successful data retrieval. Identifying and fixing errors improves query reliability, enhances database performance, and is an essential skill for database developers and administrators.

Q4. Fix Errors in an SQL Query Using JOIN Where the Join Condition is Missing or Incorrect

1. Introduction

A JOIN is an SQL operation used to combine data from two or more tables based on a common column. It helps retrieve related information stored in different tables without duplicating data. If the join condition is missing or incorrect, SQL may return incorrect results, duplicate rows, or a Cartesian Product.

2. Definition of JOIN

A JOIN combines rows from two or more tables using a common field.

General Syntax

```
SELECT column_name(s)
FROM Table1
JOIN Table2
ON Table1.common_column = Table2.common_column;
```

3. Types of SQL JOIN

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL OUTER JOIN
5. CROSS JOIN
6. SELF JOIN

4. Sample Tables

Student Table

StudentID	StudentName	DepartmentID
101	Rahul	D01
102	Priya	D02
103	Asha	D01

DepartmentID	DepartmentName
D01	Computer Science
D02	Electronics

5. Incorrect SQL Query

```
SELECT *
FROM Student
JOIN Department;
```

6. Error Identification

Error 1

Missing ON clause.

Error 2

No relationship is specified between the tables.

Error 3

Produces a Cartesian Product, where every student is matched with every department.

7. Correct SQL Query

```
SELECT Student.StudentID,  
       Student.StudentName,  
       Department.DepartmentName  
FROM Student  
JOIN Department  
ON Student.DepartmentID = Department.DepartmentID;
```

8. Explanation

SELECT

Retrieves required columns.

FROM Student

Specifies the first table.

JOIN Department

Joins the Department table.

ON Student.DepartmentID = Department.DepartmentID

Matches students with their respective departments.

9. Expected Output

StudentID	StudentName	DepartmentName
101	Rahul	Computer Science
102	Priya	Electronics
103	Asha	Computer Science

10. Cartesian Product (Incorrect Result)

Without the ON clause:

Student Department

Rahul Computer Science

Rahul Electronics

Priya Computer Science

Priya Electronics

Asha Computer Science

Asha Electronics

This incorrect result contains unnecessary combinations.

11. Advantages of Using JOIN

- Combines related data from multiple tables.
- Reduces data redundancy.
- Improves database normalization.

- Produces meaningful reports.
 - Makes querying efficient.
-

12. Real-Life Applications

- Student–Department databases.
 - Customer–Order systems.
 - Employee–Department management.
 - Hospital patient records.
 - Banking account management.
 - E-commerce order processing.
-

13. Common JOIN Errors

- Missing ON clause.
 - Wrong join column.
 - Joining unrelated tables.
 - Selecting ambiguous column names.
 - Using incorrect JOIN type.
-

15. Conclusion

JOIN is an essential SQL feature for retrieving related information from multiple tables. Specifying the correct join condition ensures accurate results, eliminates unnecessary records, and improves query efficiency.

Q5. Identify and Correct a Subquery Error Where Multiple Values are Returned Instead of a Single Value

1. Introduction

A subquery (nested query) is an SQL query written inside another SQL query. The inner query executes first, and its result is used by the outer query. Errors occur when a subquery returns multiple rows but the outer query expects only one value.

2. Definition of Subquery

A subquery is a query placed inside another SQL statement.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name operator
(
    SELECT column_name
    FROM table_name
);
```

3. Types of Subqueries

- Single-row subquery
- Multiple-row subquery
- Multiple-column subquery
- Correlated subquery
- Nested subquery

4. Sample Employee Table

EmpID Name Department Salary

101	Ram	IT	60000
102	John	IT	50000
103	Asha	HR	50000
104	Priya	Sales	45000

5. Incorrect SQL Query

```
SELECT Name
FROM Employee
WHERE Salary =
(
    SELECT Salary
    FROM Employee
    WHERE Department='IT'
);
```

6. Error Identification

The inner query returns:

60000

50000

Since two values are returned, the "=" operator cannot compare one value with multiple values.

SQL displays an error similar to:

```
Subquery returns more than one row.
```

7. Correct Method 1 (Using IN)


```
SELECT Name
FROM Employee
WHERE Salary IN
(
SELECT Salary
FROM Employee
WHERE Department='IT'
);
```

8. Output

Name

Ram

John

Asha

(Asha has salary 50000, which matches one of the IT salaries.)

9. Correct Method 2 (Using MAX)

```
SELECT Name
FROM Employee
WHERE Salary =
(
SELECT MAX(Salary)
FROM Employee
WHERE Department='IT'
);
```

10. Output

Name

Ram

11. Explanation

"=" Operator

Used only when the subquery returns one value.

IN Operator

Used when multiple values are returned.

MAX()

Converts multiple rows into one value.

12. Advantages of Subqueries

- Simplifies complex queries.
- Improves readability.
- Avoids temporary tables.
- Retrieves dynamic data.

- Supports nested calculations.
-

13. Applications

- Highest-paid employee.
 - Students scoring above average.
 - Product price comparisons.
 - Banking transaction analysis.
 - Hospital patient reports.
-

14. Common Errors

- Using "=" for multiple rows.
 - Missing parentheses.
 - Incorrect column references.
 - Returning multiple columns instead of one.
 - Wrong aggregate function usage.
-

16. Conclusion

Subqueries are powerful tools for solving complex SQL problems. Choosing the correct operator, such as IN for multiple values or aggregate functions for a single value, ensures error-free execution and accurate results.

Q6. Debug a VIEW Creation Statement That Contains Invalid Column References or Syntax Errors

1. Introduction

A **VIEW** is a virtual table created from one or more database tables. It stores only the SQL query and displays data whenever it is accessed. Views improve security, simplify complex queries, and provide customized data to users.

2. Definition of VIEW

A VIEW is a virtual table based on the result of an SQL query.

Syntax

```
CREATE VIEW view_name AS
SELECT column_name
FROM table_name;
```

3. Advantages of VIEW

- Improves security.
- Hides sensitive columns.
- Simplifies complex SQL queries.

- Reduces query repetition.
- Provides customized data access.

4. Sample Student Table

StudentID	StudentName	Age	Department
101	Rahul	20	CSE
102	Priya	21	ECE
103	Asha	22	CSE

5. Incorrect VIEW Statement

```
CREATE VIEW StudentView AS
SELECT StudentName,
Marks
FROM Student;
```

6. Error Identification

Error 1

Column **Marks** does not exist in the Student table.

Error 2

SQL cannot create a view with invalid column references.

Typical error message:

```
Unknown column 'Marks'
```

7. Correct VIEW Statement

```
CREATE VIEW StudentView AS
SELECT StudentID,
       StudentName,
       Age,
       Department
FROM Student;
```

8. Viewing the Data

```
SELECT *
FROM StudentView;
```

9. Output

StudentID	StudentName	Age	Department
101	Rahul	20	CSE
102	Priya	21	ECE
103	Asha	22	CSE

10. Explanation

CREATE VIEW

Creates a virtual table.

SELECT

Chooses the required columns.

FROM Student

Specifies the source table.

Only existing columns can be included in the view.

11. Applications of VIEW

- Employee salary reports.
 - Student information systems.
 - Banking account summaries.
 - Hospital patient records.
 - E-commerce order reports.
 - College administration.
-

12. Common Errors in VIEW Creation

- Invalid column names.
 - Incorrect table names.
 - Missing SELECT statement.
 - Syntax errors.
 - Missing permissions.
 - Duplicate view names.
-

13. Advantages of Using VIEW

- Enhances data security.
 - Simplifies complex SQL statements.
 - Hides unnecessary columns.
 - Supports data abstraction.
 - Makes reporting easier.
-

14. Conclusion

Views provide a secure and efficient way to access database information without exposing the underlying tables. Ensuring that all referenced columns exist and that the SQL syntax is correct prevents view creation errors and improves database usability.

Q7. Examine an SQL Query with Unnecessary Conditions and Optimize it by Simplifying the WHERE Clause

1. Introduction

Query optimization is the process of improving the performance and efficiency of SQL queries. An optimized query retrieves the required data using fewer resources and executes faster. One common optimization technique is removing unnecessary or redundant conditions from the **WHERE** clause.

A simplified query is easier to understand, maintain, and execute while producing the same result.

2. Definition of Query Optimization

Query Optimization is the process of selecting the most efficient method of executing an SQL query to reduce execution time and improve database performance.

3. Purpose of the WHERE Clause

The **WHERE** clause filters rows based on specified conditions.

Syntax

```
SELECT column_name(s)
FROM table_name
WHERE condition;
```

4. Incorrect SQL Query

```
SELECT *
FROM Employee
WHERE Salary > 30000
AND Salary > 20000
AND Department='HR';
```

5. Error Identification

Error 1

The condition:

Salary > 20000

is unnecessary because:

Salary > 30000

already includes all salaries greater than 20,000.

Error 2

Redundant conditions increase query complexity and may slightly affect performance.

6. Correct SQL Query

```
SELECT *
FROM Employee
WHERE Salary > 30000
AND Department='HR';
```

7. Sample Employee Table

EmpID	Name	Department	Salary
-------	------	------------	--------

101	Rahul	HR	35000
102	Priya	HR	28000
103	John	IT	45000
104	Asha	HR	50000
105	David	Sales	38000

8. Output

EmpID	Name	Department	Salary
-------	------	------------	--------

101	Rahul	HR	35000
104	Asha	HR	50000

9. Explanation

Salary > 30000

Retrieves employees earning more than ₹30,000.

Department='HR'

Filters only HR department employees.

Combined Condition

Returns only HR employees whose salary exceeds ₹30,000.

10. Query Optimization Techniques

- Remove redundant conditions.
 - Select only required columns instead of using SELECT *.
 - Use indexes on frequently searched columns.
 - Avoid unnecessary subqueries.
 - Use proper JOIN conditions.
 - Filter records as early as possible.
 - Use EXISTS instead of IN for large datasets where appropriate.
-

11. Benefits of Query Optimization

- Faster execution.
 - Lower CPU usage.
 - Reduced memory consumption.
 - Efficient database performance.
 - Better scalability.
 - Faster report generation.
-

12. Real-Life Applications

- Banking transaction systems.
 - Employee payroll.
 - Student databases.
 - Hospital management.
 - Online shopping websites.
 - Railway reservation systems.
-

13. Common Optimization Mistakes

- Using redundant conditions.
 - Using SELECT * unnecessarily.
 - Missing indexes.
 - Unnecessary nested queries.
 - Incorrect JOIN conditions.
 - Functions on indexed columns in WHERE clause.
-

15. Conclusion

Query optimization improves SQL performance by eliminating unnecessary conditions and simplifying queries. Efficient SQL queries consume fewer system resources, execute faster, and enhance overall database performance.

Q8. Identify Violations of Integrity Constraints in a Given Table and Correct Them

1. Introduction

Integrity constraints are rules applied to database tables to maintain the **accuracy, consistency, and reliability** of data. They prevent invalid data entry and ensure relationships between tables remain correct.

If integrity constraints are violated, the database may contain duplicate, incomplete, or inconsistent records.

2. Definition of Integrity Constraints

Integrity constraints are rules enforced by the DBMS to ensure only valid and consistent data is stored in a database.

3. Types of Integrity Constraints

1. Primary Key Constraint

- Uniquely identifies each record.
- Cannot contain NULL values.
- No duplicate values allowed.

2. Foreign Key Constraint

- Maintains relationships between tables.
- Prevents invalid references.

3. NOT NULL Constraint

- Prevents NULL values.

4. UNIQUE Constraint

- Prevents duplicate values.

5. CHECK Constraint

- Restricts values based on a condition.

6. DEFAULT Constraint

- Assigns a default value when none is provided.

4. Sample Table with Violations

StudentID	StudentName	Email	Age
101	Rahul	rahul@gmail.com	20
101	Priya	priya@gmail.com	21
NULL	Asha	asha@gmail.com	22
104	David	rahul@gmail.com	17

5. Error Identification

Error 1 – Duplicate Primary Key

StudentID **101** appears twice.

Violation:

- PRIMARY KEY constraint.

Error 2 – NULL Primary Key

StudentID cannot be NULL.

Violation:

- NOT NULL constraint.

Error 3 – Duplicate Email

Two students have the same email.

Violation:

- UNIQUE constraint.

Error 4 – Invalid Age

Suppose the rule is:

CHECK (Age >= 18)

Age **17** violates the CHECK constraint.

6. Correct Table

StudentID	StudentName	Email	Age
101	Rahul	rahul@gmail.com	20
102	Priya	priya@gmail.com	21
103	Asha	asha@gmail.com	22
104	David	david@gmail.com	19

7. SQL Statement with Constraints

```
CREATE TABLE Student
(
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Age INT CHECK (Age >= 18)
);
```

8. Explanation

PRIMARY KEY

Ensures every StudentID is unique.

NOT NULL

Ensures StudentName is always entered.

UNIQUE

Ensures no two students have the same email.

CHECK

Ensures only students aged 18 or above are stored.

9. Advantages of Integrity Constraints

- Prevent duplicate records.
 - Maintain accurate data.
 - Improve data consistency.
 - Enforce business rules.
 - Maintain relationships between tables.
 - Reduce data entry errors.
-

10. Real-Life Applications

- Banking systems.
 - Hospital databases.
 - University management.
 - Railway reservation.
 - Library management.
 - Online shopping websites.
-

11. Consequences of Constraint Violations

- Duplicate records.
 - Incorrect reports.
 - Loss of data consistency.
 - Invalid relationships.
 - Difficulty in database maintenance.
 - Increased chances of application errors.
-

12. Best Practices

- Always define a Primary Key.
- Use Foreign Keys for related tables.

- Apply CHECK constraints for validation.
 - Avoid unnecessary NULL values.
 - Use UNIQUE for important fields like email or phone number.
-

14. Conclusion

Integrity constraints are essential for maintaining the quality, consistency, and reliability of database systems. By enforcing rules such as **PRIMARY KEY**, **FOREIGN KEY**, **NOT NULL**, **UNIQUE**, and **CHECK**, organizations can prevent invalid data entry, reduce errors, and ensure efficient database management. Proper use of integrity constraints is a fundamental practice in designing secure and reliable relational databases.

Q9. Fix a Relation that Violates 1NF or 2NF by Removing Repeating Groups or Partial Dependencies

1. Introduction

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It divides large tables into smaller, related tables while maintaining relationships between them. The first two stages of normalization are **First Normal Form (1NF)** and **Second Normal Form (2NF)**.

Normalization prevents data anomalies and improves database efficiency.

2. Definition of Normalization

Normalization is a database design technique used to eliminate redundant data and ensure data dependencies are logical.

3. Objectives of Normalization

- Eliminate duplicate data.
 - Improve data consistency.
 - Reduce storage space.
 - Avoid insertion, update, and deletion anomalies.
 - Improve database performance.
-

4. Normal Forms

First Normal Form (1NF)

- Each column contains only **atomic (single)** values.
- No repeating groups.
- Each row is unique.

Second Normal Form (2NF)

- Table must already be in **1NF**.
- Removes **partial dependency**.
- Every non-key attribute depends on the entire primary key.

5. Unnormalized Relation (UNF)

StudentID StudentName Subjects

101	Rahul	DBMS, Java
102	Priya	Python, CN

Problem

The **Subjects** column contains multiple values.

This violates **1NF** because one attribute should contain only one value.

6. Convert to First Normal Form (1NF)

StudentID StudentName Subject

101	Rahul	DBMS
101	Rahul	Java
102	Priya	Python
102	Priya	CN

Explanation

- Each cell contains only one value.
- Repeating groups are removed.
- Data becomes easier to query.

7. Convert to Second Normal Form (2NF)

Student Table

StudentID StudentName

101	Rahul
102	Priya

Enrollment Table

StudentID Subject

101	DBMS
101	Java
102	Python
102	CN

Explanation

StudentName depends only on StudentID, not on Subject. Therefore, StudentName is separated into another table.

8. SQL Statements

Student Table

```
CREATE TABLE Student
(
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50)
);
```

Enrollment Table

```
CREATE TABLE Enrollment
(
    StudentID INT,
    Subject VARCHAR(50),
    FOREIGN KEY(StudentID)
    REFERENCES Student(StudentID)
);
```

9. Advantages of 1NF and 2NF

- Eliminates duplicate data.
- Improves consistency.
- Reduces storage requirements.
- Prevents update anomalies.
- Easier database maintenance.
- Improves query performance.

10. Data Anomalies Prevented

Insertion Anomaly

Cannot insert unnecessary duplicate information.

Update Anomaly

Updating data in one place automatically maintains consistency.

Deletion Anomaly

Deleting one record does not remove unrelated information.

11. Applications

- College Management System
- Hospital Database
- Banking System
- Railway Reservation
- Library Management
- Employee Database

13. Conclusion

Normalization improves the quality of database design by eliminating redundancy and ensuring data integrity. Converting a relation into **1NF** removes repeating groups, while **2NF** removes partial dependencies, making the database efficient, consistent, and easier to maintain.

Q10. Identify Redundant Attributes in a Relation and Apply Normalization (Up to 3NF) to Improve Database Design

1. Introduction

A well-designed database minimizes redundant data and stores information efficiently. **Third Normal Form (3NF)** removes **transitive dependencies**, where one non-key attribute depends on another non-key attribute instead of the primary key. Applying normalization up to 3NF improves database consistency, reduces duplication, and simplifies maintenance.

2. Definition of Third Normal Form (3NF)

A relation is in **Third Normal Form (3NF)** if:

1. It is already in **Second Normal Form (2NF)**.
 2. There are **no transitive dependencies**.
 3. Every non-key attribute depends only on the primary key.
-

3. Original Relation

StudentID	StudentName	DeptID	DeptName	HOD
101	Rahul	D01	Computer Science	Dr. Rao
102	Priya	D02	Electronics	Dr. Kumar
103	Asha	D01	Computer Science	Dr. Rao

4. Redundant Attributes

The following attributes are repeated for every student in the same department:

- DeptName
- HOD

This causes data redundancy.

5. Problems Caused by Redundancy

- Wastes storage space.
- Difficult to update department information.
- Increased chances of inconsistent data.
- Slower database performance.

6. First Normal Form (1NF)

The relation already satisfies 1NF because all values are atomic.

7. Second Normal Form (2NF)

Since StudentID is the primary key, every non-key attribute depends on it. However, DeptName and HOD depend on DeptID rather than StudentID.

8. Third Normal Form (3NF)

Student Table

StudentID StudentName DeptID

101	Rahul	D01
102	Priya	D02
103	Asha	D01

Department Table

DeptID DeptName HOD

D01	Computer Science	Dr. Rao
D02	Electronics	Dr. Kumar

9. SQL Statements

Department Table

```
CREATE TABLE Department
(
    DeptID VARCHAR(5) PRIMARY KEY,
    DeptName VARCHAR(50),
    HOD VARCHAR(50)
);
```

Student Table

```
CREATE TABLE Student
(
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    DeptID VARCHAR(5),
    FOREIGN KEY(DeptID)
    REFERENCES Department(DeptID)
);
```

10. Advantages of 3NF

- Eliminates transitive dependency.
- Removes redundant data.
- Improves consistency.
- Saves storage space.
- Simplifies updates.
- Prevents anomalies.
- Improves database performance.

12. Applications

- University databases
- Banking systems
- Hospital management
- Railway reservation
- Inventory management
- Employee management
- Library management
- Online shopping systems

13. Comparison of Normal Forms

Normal Form	Purpose	Removes
1NF	Atomic values	Repeating groups
2NF	Full dependency	Partial dependency
3NF	No transitive dependency	Redundant attributes

15. Conclusion

Normalization up to **Third Normal Form (3NF)** creates an efficient and well-structured database by removing redundant attributes and transitive dependencies. It improves data integrity, minimizes storage requirements, prevents insertion, update, and deletion anomalies, and enhances database performance. Proper normalization is a fundamental practice in designing reliable and scalable relational database systems suitable for real-world applications.

